SIMATS ENGINEERING

ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0515** **Slot B**

COURSE OUTCOME COVERED

CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 20%

QUESTIONS:5 **Total Marks 20** **Duration :60 Minutes**

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I M. Pranay (192525382)_certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M. Pranay

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1. Real-Time Object Detection Pipeline

A system must:

A. Capture video frames
B. Detect objects using a trained model
C. Draw bounding boxes and labels
D. Display results in real time

Constraints:

- Must process frames efficiently
- Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

2. Image Difference Detection System

A system must:

A. Read two images
B. Compare them
C. Highlight differences

Constraints:

- Handle noise
- Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

3. Video Frame Extraction System

A system must:

A. Read video input
B. Extract frames at fixed intervals
C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

4. Background Subtraction Pipeline

A surveillance system must:

A. Read video stream
B. Separate foreground from background
C. Highlight moving objects

Constraints:

- Handle dynamic background
- Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

5. Edge Detection Pipeline for Real-Time Video

A system must:

A. Capture video frames

B. Apply edge detection
C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

ANSWERS

1. Real-Time Object Detection Pipeline

Pseudocode:

```
BEGIN

INPUT: Live video stream
OUTPUT: Video with detected objects, bounding boxes and labels

FUNCTION LOAD_MODEL()
   Load trained object detection model
   Return model
END FUNCTION

FUNCTION PREPROCESS(frame)
   Resize frame to required size
   Normalize frame if required
   Return processed frame
END FUNCTION

FUNCTION DETECT_OBJECTS(model, frame)
   Preprocess frame
   Send frame to trained model
   Get object classes, confidence scores and bounding boxes
   Remove detections below confidence threshold
   Return valid detections
END FUNCTION

FUNCTION VISUALIZE(frame, detections)
   FOR each detection
      Draw bounding box on frame
      Draw object label and confidence score
   END FOR
   Return frame
END FUNCTION

model ← LOAD_MODEL()
video ← OPEN_VIDEO_CAMERA()

WHILE video is available
   frame ← READ_FRAME(video)
```

```
    IF frame is not available
        BREAK
    END IF

    processed_frame ← PREPROCESS(frame)
    detections ← DETECT_OBJECTS(model, processed_frame)

    result ← VISUALIZE(frame, detections)

    DISPLAY result

    IF user presses 'q'
        BREAK
    END IF
END WHILE

RELEASE video
CLOSE display

END
```

Justification:

- Modular functions make the algorithm easy to maintain.
- Confidence threshold removes incorrect detections.
- Frame resizing improves real-time performance.
- The loop supports multiple objects in every frame.

2. Image Difference Detection System

Pseudocode:

```
BEGIN

INPUT: Image 1, Image 2
OUTPUT: Image highlighting differences

FUNCTION READ_IMAGES()
    image1 ← Read first image
    image2 ← Read second image
    Return image1, image2
END FUNCTION

FUNCTION PREPROCESS(image)
    Convert image to grayscale
    Apply Gaussian blur to reduce noise
    Return processed image
END FUNCTION

FUNCTION COMPARE_IMAGES(image1, image2)
    difference ← Absolute difference between image1 and image2
    Apply threshold to difference image
    Apply morphological operation to remove noise
```

```
   Return difference
END FUNCTION

FUNCTION HIGHLIGHT_DIFFERENCES(original, difference)
   Find contours in difference image

   FOR each contour
      IF contour area > minimum area
         Draw bounding box around difference
      END IF
   END FOR

   Return highlighted image
END FUNCTION

image1, image2 ← READ_IMAGES()

image1 ← PREPROCESS(image1)
image2 ← PREPROCESS(image2)

IF image dimensions are different
   Resize image2 to image1 dimensions
END IF

difference ← COMPARE_IMAGES(image1, image2)

result ← HIGHLIGHT_DIFFERENCES(image1, difference)

DISPLAY result

SAVE result

END
```

Justification:

- Grayscale reduces computational complexity.
- Gaussian blur reduces noise.
- Absolute difference identifies changed regions.
- Thresholding separates significant changes from small variations.
- Morphological filtering improves accuracy.

3. Video Frame Extraction System
Pseudocode:

```
BEGIN

INPUT: Video file
OUTPUT: Selected frames saved as images

FUNCTION OPEN_VIDEO()
   Open video file
   Get total frames and frame rate
   Return video
END FUNCTION

FUNCTION SAVE_FRAME(frame, frame_number)
   Create unique filename using frame number
   Save frame to storage
END FUNCTION

video ← OPEN_VIDEO()

interval ← FIXED FRAME INTERVAL
frame_count ← 0
saved_count ← 0

WHILE video is available

   frame ← READ_FRAME(video)

   IF frame is not available
      BREAK
   END IF

   frame_count ← frame_count + 1

   IF frame_count MOD interval = 0

      IF current frame is not similar to previous saved frame
         SAVE_FRAME(frame, frame_count)
         saved_count ← saved_count + 1
      END IF

   END IF

END WHILE

RELEASE video

DISPLAY number of saved frames

END
```

Justification:

- Frames are extracted at fixed intervals.

- Only selected frames are stored, reducing storage requirements.
- Similar-frame checking avoids redundant frames.
- Sequential frame processing improves efficiency.

4. Background Subtraction Pipeline
Pseudocode:

```
BEGIN

INPUT: Live surveillance video
OUTPUT: Video showing moving foreground objects

FUNCTION INITIALIZE_BACKGROUND()
   Create background subtraction model
   Set parameters for dynamic background
   Return background model
END FUNCTION

FUNCTION PREPROCESS(frame)
   Resize frame
   Apply Gaussian blur
   Return processed frame
END FUNCTION

FUNCTION REMOVE_NOISE(foreground)
   Apply thresholding
   Apply morphological opening
   Apply morphological closing
   Return cleaned foreground
END FUNCTION

FUNCTION DETECT_OBJECTS(foreground)
   Find contours

   FOR each contour
      IF contour area > minimum area
         Calculate bounding box
         Store bounding box
      END IF
   END FOR

   Return detected objects
END FUNCTION

background_model ← INITIALIZE_BACKGROUND()
video ← OPEN_VIDEO_CAMERA()

WHILE video is available
```

```
    frame ← READ_FRAME(video)

    IF frame is not available
        BREAK
    END IF

    processed_frame ← PREPROCESS(frame)

    foreground ← APPLY_BACKGROUND_SUBTRACTION(
                    background_model,
                    processed_frame)

    foreground ← REMOVE_NOISE(foreground)

    objects ← DETECT_OBJECTS(foreground)

    FOR each object
        Draw bounding box on frame
    END FOR

    DISPLAY frame
    DISPLAY foreground

    IF user presses 'q'
        BREAK
    END IF

END WHILE

RELEASE video
CLOSE display

END
```

Justification:

- Background modeling separates moving objects from the scene.
- Adaptive background subtraction helps with dynamic backgrounds.
- Gaussian blur reduces camera and sensor noise.
- Morphological operations remove small unwanted regions.
- Contour-area filtering reduces false detections.

5. Edge Detection Pipeline for Real-Time Video
Pseudocode:

```
BEGIN

INPUT: Live video stream
OUTPUT: Video displaying detected edges

FUNCTION CAPTURE_FRAME(video)
   frame ← READ_FRAME(video)
   Return frame
END FUNCTION

FUNCTION PREPROCESS(frame)
   Resize frame for real-time processing
   Convert frame to grayscale
   Apply Gaussian blur
   Return processed frame
END FUNCTION

FUNCTION DETECT_EDGES(frame)
   edges ← Apply Canny edge detection
   Return edges
END FUNCTION

FUNCTION VISUALIZE(edges)
   Display edge image
END FUNCTION

video ← OPEN_VIDEO_CAMERA()

WHILE video is available

   frame ← CAPTURE_FRAME(video)

   IF frame is not available
      BREAK
   END IF

   processed_frame ← PREPROCESS(frame)

   edges ← DETECT_EDGES(processed_frame)

   VISUALIZE(edges)

   IF user presses 'q'
      BREAK
   END IF

END WHILE

RELEASE video
CLOSE display
```

END

Justification:

- Resizing reduces processing time and supports real-time operation.
- Grayscale simplifies edge detection.
- Gaussian blur removes noise before detecting edges.
- Canny provides clear and well-defined edges.
- Continuous frame processing allows real-time video analysis.